📝 Minggu 6 – REST API di Laravel

Materi Teori

- **REST API**: Antarmuka komunikasi antara client (browser/mobile/Postman) dengan server.
- **HTTP Method CRUD**:
  - **GET** → ambil data (index/show)
  - **POST** → tambah data (store)
  - **PUT/PATCH** → update data
  - **DELETE** → hapus data
- **Struktur API di Laravel**:
  - **Route** → `routes/api.php`
  - **Controller** → `php artisan make:controller NamaController --api`
  - **Response JSON** → default format API.

Langkah Praktik

1️⃣ Buat Controller API

```
php artisan make:controller MemberController --api

php artisan make:controller AuthorController --api

php artisan make:controller PublisherController --api

php artisan make:controller BookController --api

php artisan make:controller LoanController --api
```

2️⃣ Tambah Route di `routes/api.php`

```
use App\Http\Controllers\MemberController;

use App\Http\Controllers\AuthorController;

use App\Http\Controllers\PublisherController;

use App\Http\Controllers\BookController;

use App\Http\Controllers\LoanController;
```

```
Route::apiResource('members', MemberController::class);
Route::apiResource('authors', AuthorController::class);
Route::apiResource('publishers', PublisherController::class);
Route::apiResource('books', BookController::class);
Route::apiResource('loans', LoanController::class);
```

3️⃣ Implementasi CRUD di Controller (contoh `MemberController`)

```
<?php

namespace App\Http\Controllers;

use App\Models\Member;
use Illuminate\Http\Request;

class MemberController extends Controller
{
    public function index() {
        return response()->json(Member::all(), 200);
    }

    public function store(Request $request) {
        $request->validate([
            'name' => 'required',
            'email' => 'required|email|unique:members',
            'phone' => 'required',
        ]);
        $member = Member::create($request->all());
        return response()->json($member, 201);
```

```
    }

    public function show($id) {
        $member = Member::findOrFail($id);
        return response()->json($member, 200);
    }

    public function update(Request $request, $id) {
        $member = Member::findOrFail($id);
        $member->update($request->all());
        return response()->json($member, 200);
    }

    public function destroy($id) {
        Member::destroy($id);
        return response()->json(null, 204);
    }
}
```

4 Tambahkan `fillable` di Model

Contoh di `app/Models/Member.php`:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
```

```
class Member extends Model
{
    use HasFactory;
    protected $fillable = ['name', 'email', 'phone'];
}
```

Lakukan hal yang sama untuk:

- **Author.php** → `['name']`
- **Publisher.php** → `['name']`
- **Book.php** → `['title','author_id','publisher_id']`
- **Loan.php** → `['member_id','book_id','loan_date','return_date']`

5️⃣ Test API di Postman

Contoh request:

- GET http://127.0.0.1:8000/api/members

- POST http://127.0.0.1:8000/api/members

```
{
  "name": "Siti",
  "email": "siti@mail.com",
  "phone": "081222333"
}
```

- PUT http://127.0.0.1:8000/api/members/1

- DELETE http://127.0.0.1:8000/api/members/1

📌 Tugas / Milestone

1. Mahasiswa membuat API untuk **Anggota, Buku, Author, Penerbit, dan Peminjaman**.
2. Semua endpoint CRUD bisa dites di Postman dengan response JSON valid.
3. Screenshoot hasil **GET, POST, PUT, DELETE** di Postman → dikumpulkan sebagai laporan.